

PROGRAMMATION FONCTIONNELLE 2 : TD7 (ARBRES, ENTIERS EN BINAIRE)

mars 2026 — Pierre Rousselin

Exercice 1 : arbres binaires

On rappelle la définition des arbres binaires (non stricts) donnée en cours :

```
Inductive btree (A : Type) :=
| emptyTree
| node (left : btree A) (key : A) (right : btree A).
```

1. Définir les fonctions `height` (hauteur d'un arbre, la hauteur de l'arbre vide est 0) et `numNodes` (nombre de sommets).
2. Définir l'opération `mirror` qui change gauche en droite dans tout l'arbre.
3. Donner la preuve en Rocq, puis la « preuve papier très détaillée » du lemme suivant :
`Lemma numNodes_mirror {A : Type} (t : btree A) : numNodes (mirror t) = numNodes t.`
 De quels résultats supplémentaires avez-vous eu besoin ?
4. Définir la fonction `prefixDFS : btree A -> list A` qui donne la liste de toutes les clés dans l'arbre en commençant par la racine, puis en parcourant le sous-arbre gauche et enfin le sous-arbre droit.
5. Que dire, pour un arbre binaire `t` de `length (prefixDFS t)` ? Le prouver avec une « preuve papier très détaillée » et/ou une preuve en Rocq. De quels résultats supplémentaires avez-vous eu besoin ?
6. Un arbre binaire parfait est un arbre dont tous les niveaux sont pleins et toutes les feuilles sont à la même hauteur. Définir la fonction `perfect : nat -> A -> btree A` qui construit un tel arbre, dont la hauteur est le premier argument et dont toutes les clés sont égales à son deuxième argument.
7. Prouver que `perfect n` a bien hauteur `n`.
8. Définir la puissance d'un entier par un autre entier, puis donner la preuve papier très détaillée de :
`Lemma numNodes_perfect {A : Type} (n : nat) (a : A) :`
`S (numNodes (perfect n a)) = 2 ^ n.`
 où l'opérateur `^` désigne la puissance.

--- * ---

Correction de l'exercice 1 :

1. `Fixpoint height {A : Type} (t : btree A) :=`
`match t with`
`| emptyTree => 0`
`| node l _ r => S (max (height l) (height r))`
`end.`
`Fixpoint numNodes {A : Type} (t : btree A) :=`
`match t with`
`| emptyTree => 0`
`| node l _ r => S (numNodes l + numNodes r)`
`end.`
2. `Fixpoint mirror {A : Type} (t : btree A) :=`
`match t with`
`| emptyTree => emptyTree`
`| node l k r => node (mirror r) k (mirror l)`
`end.`
3. Preuve en Rocq (pas le temps cette fois-ci de faire la preuve papier) :
`Lemma numNodes_mirror {A : Type} (t : btree A) :`
`numNodes (mirror t) = numNodes t.`
`Proof.`
`induction t as [| l IHl k r IHr].`
`- reflexivity.`
`- simpl. rewrite IHl IHr add_comm. reflexivity.`
`Qed.`

```

4. Fixpoint prefixDFS {A : Type} (t : btree A) :=
  match t with
  | emptyTree => []
  | node l k r => k :: (prefixDFS l ++ prefixDFS r)
  end.

5. Lemma length_prefixDFS {A : Type} (t : btree A) :
  length (prefixDFS t) = numNodes t.
Proof.
  induction t as [| l IHl k r IHr].
  - reflexivity.
  - simpl.
    rewrite length_app IHl IHr.
    reflexivity.
Qed.

6. Fixpoint perfect {A : Type} (n : nat) (a : A) :=
  match n with
  | 0 => emptyTree
  | S p => node (perfect p a) a (perfect p a)
  end.

7. Lemma height_perfect {A : Type} (n : nat) (a : A) :
  height (perfect n a) = n.
Proof.
  induction n as [| p IH].
  - reflexivity.
  - simpl. rewrite IH. lia.
Qed.

8. Lemma numNodes_perfect {A : Type} (n : nat) (a : A) :
  S (numNodes (perfect n a)) = 2 ^ n.
Proof.
  induction n as [| p IH].
  - rewrite perfect_0 numNodes_empty pow_0_r. reflexivity.
  - simpl. rewrite -add_succ_l -add_succ_r IH add_0_r. reflexivity.
Qed.

```

--- * ---

Exercice 2 : Arbres binaires stricts

On rappelle la définition des arbres binaires stricts :

```

Inductive sbtree (A : Type) :=
  | leaf (key : A)
  | node (left : sbtree A) (key : A) (right : sbtree A).

```

1. Rappeler la définition des fonctions `numInnerNodes` (nombre de nœuds internes) et `numLeaves` (nombre de feuilles).
2. Quelle relation y a-t-il entre ces deux valeurs ? Énoncer un lemme en Rocq et en donner la preuve.
3. Cette relation est-elle encore valable pour des arbres binaires non stricts ?
4. Écrire une fonction qui permet d'encoder un `btree A` dans un `sbtree (option A)`, où `option` est le type


```

Inductive option {A : Type} := Some (a : A) | None.

```

 Cette fonction est-elle bijective ?

--- * ---

Correction de l'exercice 2 :

1. **Fixpoint** numInnerNodes {A : Type} (t : sbtree A) :=
`match t with`
`| leaf _ => 0`
`| node l _ r => S (numInnerNodes l + numInnerNodes r)`
`end.`
Fixpoint numLeaves {A : Type} (t : sbtree A) :=
`match t with`
`| leaf _ => 1`
`| node l _ r => numLeaves l + numLeaves r`
`end.`
2. **Lemma** numLeaves_numInnerNodes {A : Type} (t : sbtree A) :
`numLeaves t = S (numInnerNodes t).`
Proof.
`induction t as [k | l IHl k r IHr].`
`- reflexivity.`
`- simpl.`
`rewrite IHl IHr add_succ_r.`
`reflexivity.`
Qed.
3. Cette relation n'est plus valable dans les arbres binaires non stricts, on a seulement une inégalité.
4. **Fixpoint** to_sbtree {A : Type} (t : btree A) : SBTree.sbtree (option A) :=
`match t with`
`| emptyTree => SBTree.leaf None`
`| node l k r => SBTree.node (to_sbtree l) (Some k) (to_sbtree r)`
`end.`

Elle n'est pas bijective, les None ne sont que dans les feuilles.

--- * ---